

Lecture 1: Introduction to Parallel and Distributed Computing

1. Computing History & Moore's Law

- **Early Era:** ENIAC/UNIVAC used vacuum tubes, taking up entire rooms. Mainframe era followed with IBM System/360 (1960s).
- **Personal Computers:** Apple II and IBM PC popularized computing.
- **Moore's Law:** Predicts transistor count on chips doubles every two years, driving processing power growth.

2. Types of Computing

- **Serial Computing:** A single CPU executes instructions one after another. Simple but slow.
- **Parallel Computing:** Multiple CPUs solve parts of one single problem at the same time.
 - **Advantages:** Faster computation, efficient use of resources.
 - **Disadvantages:** High programming complexity, synchronization issues, and higher hardware cost (more cores/GPUs).
 - **Examples:** Multi-core CPUs, GPUs, supercomputers, scientific simulation, AI/ML training.
- **Distributed Computing:** Multiple independent computers connected via a network acting as one system.
 - **Advantages:** High fault tolerance (one fails, others keep working), cost-effective scaling via commodity hardware.
 - **Disadvantages:** Network latency/overhead, complex management, security vulnerabilities.
 - **Examples:** Cloud systems (clusters), microservices, Hadoop/Spark-style clusters, IoT networks, blockchains.
- **Parallel vs Distributed:** Parallel = single computer, multiple processors, shared/distributed memory, bus communication. Distributed = multiple computers, distributed memory only, message passing over network.

3. Parallel Memory Architectures

- **Shared Memory:** All processors use one global memory. Easy to program, but poor scalability due to memory path contention.
- **Distributed Memory:** Each processor has local memory and communicates via messages. Highly scalable, but harder to program (explicit data movement).
- **Hybrid Memory:** Shared memory within a node, distributed communication across nodes. Offers best scalability and performance but maximum complexity.

4. Communication Concepts

- **Latency vs Bandwidth:** Latency is the startup delay to send a small message; bandwidth is the data transferred per unit time.
- **Synchronous vs Asynchronous:** Synchronous is blocking (task waits); asynchronous is non-blocking.
- **Point-to-Point vs Collective:** Point-to-point is between two tasks; collective involves all tasks in a group.

Lecture 2: Flynn's Taxonomy & Parallelism

1. Flynn's Taxonomy

- **Instruction Stream:** The sequence of instructions fetched/executed by a CPU (conceptually tied to a program counter).
- **Data Stream:** The sequence of data items those instructions operate on.
- **SISD:** Single Instruction, Single Data. Traditional uniprocessor (old PCs). Example: single-core sequential machine (classic Von Neumann style).
- **SIMD:** Single Instruction, Multiple Data. Exact same operation runs on different data simultaneously. Best for vectors, matrix math, GPUs.
 - **Array Processor:** Each parallel processing unit has its own separate memory and register file.
 - **Pipelined Processor:** Read data from central resource (typically register file in modern CPUs), process fragments, write results back.
 - **Associative Processor:** Each parallel processing unit independently decides whether to execute or skip (predicated/masked SIMD).
- **MISD:** Multiple Instruction, Single Data. Different operations on the exact same data stream. Rare, rarely implemented commercially. Example: $Z = \sin(x) + \cos(x) + \tan(x)$.

- **MIMD:** Multiple Instruction, Multiple Data. Entirely different instructions on different datasets. Standard for modern distributed and multi-core systems.
 - **Shared Memory MIMD (Tightly Coupled):** Connected to a single global memory. Prone to memory contention, less tolerant to failures, and harder to extend.
 - **Distributed Memory MIMD (Loosely Coupled):** Each PE has local memory; communicates via an interconnection network. More scalable, fault-tolerant.

2. SPMD vs MPMD

- **SPMD:** Single Program Multiple Data (proposed by Federica Dardes). Every node runs the exact same code, but works on different chunks of data.
- **MPMD:** Multiple Program Multiple Data. Manager-worker model where nodes run completely different programs (heterogeneous tasks).

3. Types of Parallelism

- **Hardware-Level:** Pipelining, Superscalar, VLIW, SIMD, Multi-core, Clusters.
- **Software-Level:** Instruction-Level (ILP), Data-Level (DLP), Task-Level (TLP).

4. Needs & Limitations

- **Why Needed:** Serial computation wastes hardware potential; real-world data requires faster crunching.
- **Applications:** Databases, AI/ML, Real-time systems, VR/AR.
- **Limitations:** Synchronization overhead, algorithm design complexity, portability issues.

Lecture 3: Concurrency & Processors

1. Concurrency vs Parallelism

- **Concurrency:** OS handles multiple tasks by quickly context-switching between them (illusion of simultaneous execution). Focuses on structure and scheduling.
- **Parallelism:** Tasks run physically at the exact same time on multiple processing cores.
- **Key Difference:** Concurrency addresses multiple tasks at once but not necessarily simultaneously; parallelism executes tasks at the same time using different processors/cores.

2. Motivation for Concurrent Systems

- Physical resource sharing (limited hardware for multiple users).
- Logical resource sharing (shared files/data).
- Speedup through overlapping execution and modular design.

3. Independent vs Cooperating Processes

- **Independent:** State is not shared; output is deterministic/predictable.
- **Cooperating:** State is shared; output is non-deterministic (depends on execution order).

4. Process Operations

- **Creation:** Parent creates child (execute concurrently or wait; share resources or isolate).
- **Termination:** Parent can terminate child if resources exceeded or task done. Cascading rule: if a parent is terminated by the OS, its children must also be terminated.

5. Concurrency Issues, Advantages & Drawbacks

- **Issues:** Inconsistency, loss of data, deadlocks, race conditions, starvation, blocking.
- **Advantages:** Running multiple applications, better resource utilization, vastly improved response times.
- **Drawbacks:** Difficult to coordinate, heavy overhead on OS, hard to track bugs. Limits parallelism and introduces overhead/deadlock risks with critical sections.

6. Microprocessor Types

- **Scalar:** One data element at a time.
- **Vector/Array:** Multiple data elements.

- **Superscalar:** Multiple pipelines executing more than one instruction per clock cycle for enhanced performance.

7. RISC vs CISC

- **RISC:** Simple instructions (1 cycle), fixed length, heavily pipelined, more general-purpose registers. Hardwired control. Needs more lines of code.
 - **Advantages:** Simpler instruction decoding, faster execution, lower power consumption.
 - **Disadvantages:** More instructions required for complex tasks, increased memory usage.
- **CISC:** Complex instructions (multi-cycle), variable length, fewer registers. Uses both hardwired and microprogrammed control. Less code needed.
 - **Advantages:** Reduced code size, memory-efficient, widely used with large software base.
 - **Disadvantages:** Slower execution due to complex instruction decoding, higher power consumption.
- **CPU Performance:** RISC reduces CPI but increases instructions per program; CISC reduces instructions per program but can increase CPI.

Lecture 4: Vector and Array Processors

1. Vector vs Scalar Processors

- **Scalar Processor:** Works on one data element at a time. Simpler, cheaper, lower power. Supports conditional branching natively. Applications: Video games, web browsing, office applications.
- **Vector Processor:** Works on arrays/vectors of data in one instruction. Highly parallel, expensive. Applications: CAD, virtual reality, scientific simulations.
 - **Architectural Constraints:** Vector processors do NOT support conditional vector branching easily and require multiple caches & significantly higher memory bandwidth due to bulk memory accesses.

2. Array Processors (SIMD)

- **Attached:** An auxiliary processor attached to a main host general-purpose computer with an I/O interface and a local memory interface.
- **SIMD Array:** Built-in master controller issues one single instruction to multiple parallel Processing Elements (PEs). PE typically contains: ALU, floating-point unit, and working registers.

3. Vector Processor Architectures

- **Memory-to-Memory:** Fetches operands and writes results directly back to main memory. Slower speed, but no limitation on data size.
- **Register-to-Register:** Uses dedicated internal vector registers. Much faster execution but limited by hardware register sizes.
- **Hybrid Architecture:** Combines both approaches. Provides flexible operand retrieval methods for balanced performance and efficiency.

4. Vector Instruction Format

- Contains: Opcode, Source/Destination registers, Vector length, Base address/offset, Address increment (stride), Mask field, Addressing mode field, and Data type field.

Lecture 5: Multi-Threading & Synchronization

1. Multithreading vs Multitasking

- **Multitasking:** OS running multiple different programs at once (e.g. music + browsing + typing).
- **Multithreading:** A single program executing multiple threads (e.g. Word checking spelling while typing).
- **Thread vs Process:** A process is a complete program in execution; a thread is a smaller part of a process that can run independently within that process.

2. Thread Lifecycle

- **New:** Thread created but not yet scheduled to run.
- **Runnable:** Ready to run (may be running or waiting for CPU time slice).

- **Waiting:** Blocked until some event/resource is available (e.g., lock, I/O).
- **Timed Waiting:** Waiting with a timeout/sleep.
- **Terminated:** Finished execution.
- **Lifecycle:** New → Runnable → Waiting/Timed Waiting → Terminated.

3. Benefits & Drawbacks

- **Benefits:** Better CPU utilization, faster interactive responses, better modularity.
- **Drawbacks:** Complex synchronization, deadlocks, resource contention, debugging difficulty.

4. Process Synchronization & Critical Section

- **Competitive Synchronization:** Multiple processes compete for a shared resource. Lack of sync may lead to inconsistency or data loss.
- **Cooperative Synchronization:** Multiple processes are interdependent; execution of one affects others. Lack of sync may lead to deadlock.
- **Critical Section Rules:** (1) Mutual Exclusion — only 1 process in CS at a time. (2) Progress — only waiting processes compete if section is empty. (3) Bounded Waiting — limited waiting time to enter.
- **Preemptive Kernels:** Allows a process to be preempted while running in kernel mode.
- **Non-preemptive Kernels:** A process in kernel mode runs until it exits, blocks, or voluntarily yields. Essentially free from race conditions on kernel data structures.

5. Classical IPC Problems

- **Producer-Consumer:** Buffer underflow/overflow issues.
- **Readers-Writers:** Multiple concurrent readers allowed, but writers require exclusive access.
- **Dining Philosophers:** Models deadlock and starvation.

Lecture 6: Concurrency Control (Mutex vs Semaphore)

1. Mutual Exclusion

- **Concept:** "No two processes can exist in the critical section at the same time." Essential to avoid race conditions.
- **Approaches:** Software (error-prone, high overhead), Hardware (special-purpose instructions, can cause spinlocks), OS/Language (built-in primitives like Mutexes/Semaphores).

2. Mutex

- Strict, ownership-based locking mechanism.
- Only the exact thread that acquired/locked the mutex is allowed to unlock it.
- Best for protecting a single shared resource.
- Many OSes implement priority inheritance with mutexes to reduce priority inversion impact.

3. Semaphore

- A signaling mechanism that uses an integer counter. Does not require strict ownership.
- **Wait (P):** Decrements value, blocks if unavailable. **Signal (V):** Increments value, wakes waiters.
- **Binary Semaphore:** Values 0/1 (acts similarly to a Mutex).
- **Counting Semaphore:** Values > 1. Manages a pool of identical resources.
- **Advantages:** Simple, flexible, supports coordination across multiple threads/processes, helps prevent race conditions.
- **Disadvantages:** Overhead; can deadlock if acquired/released incorrectly; hard to debug/maintain; can suffer from busy waiting/priority inversion.
- **Limitations:** Inherently causes busy waiting and priority inversion issues unless a waiting queue is explicitly implemented. Vulnerable to DOS attacks.

Lecture 7: Amdahl's Law

1. Core Concept & Formula

- Amdahl's law states that overall performance speedup is strictly limited by the remaining sequential (non-parallelizable) portion of the code.
- Formula:** $S = 1 / (\alpha + (1-\alpha)/q)$ where α = serial fraction, q = improvement factor.
- Example:** Even with infinite processors, if 10% of a program is sequential ($\alpha = 0.1$), max speedup = $1/0.1 = 10x$.

2. Key Implications

- Golden Rule:** Optimize the most frequently executed parts of the code. ("Make the common case fast").
- Removing bottlenecks in rarely used code provides minimal global benefit.

3. Scaled Amdahl's Law

- As computing power increases, users tend to run larger data problem sizes rather than just solving small problems faster. Highlights why parallel systems are powerful at scale.

Lecture 8: Pipelining Architecture

1. What is Pipelining?

- A technique where multiple instruction stages are overlapped in an assembly-line fashion to boost CPU throughput.
- Advantages:** Vastly increases instruction throughput and allows higher CPU clock speeds.
- Disadvantages:** Increases hardware complexity, triggers hazards, raises power consumption.

2. Types of Pipelining

- Hardware:** Dedicated physical segments. Includes Instruction pipelining, Arithmetic pipelining (ALUs), and Data pipelining (DMA).
- Software:** Compiler optimizations. Includes Loop Unrolling, VLIW scheduling, Speculative Execution, and Software Parallelism in Multi-Core Processors (OpenMP/CUDA).

3. The 5 Standard Pipeline Stages

- Fetch (IF):** Grab the instruction from memory.
- Decode (ID):** Translate what the instruction means.
- Execute (EX):** Perform the ALU mathematical operation.
- Memory (MEM):** Read or write data to main memory if needed.
- Write Back (WB):** Save the final result back into the CPU registers.

4. Pipeline Hazards & Mitigation

- Data Hazards:** True dependency (Instruction 2 needs output of Instruction 1). Mitigation: Operand Forwarding, Register Renaming.
 - RAW** (Read After Write — true dependency), **WAR** (Write After Read — anti-dependency), **WAW** (Write After Write — output dependency).
- Structural Hazards:** Hardware conflict (two instructions need the same executing unit). Mitigation: Resource duplication.
- Control Hazards:** Branching/Jumping confuses fetch order. Mitigation: Branch Prediction, Speculative execution.

5. Pipeline Performance Theory

- CPI:** Under ideal pipelining, Clock Cycles Per Instruction (CPI) approaches 1.
- Latency vs Throughput:** Pipelining massively improves throughput, though single-instruction latency slightly increases.
- Space-Time Diagram:** A theoretical graphical representation used to visualize overlapping instruction execution across pipeline stages over multiple clock cycles. Maps stages (Space) against clock cycles (Time).